

Projeto Integrador com Docker e Nuvem

NuvemLab - aplicação institucional híbrida com FastAPI, Docker e Azure App Service

Equipe _____	Turma _____	Data ____/____/____
-----------------	----------------	------------------------

Resumo executivo

O NuvemLab combina uma página responsiva com uma API REST real. A solução publica o catálogo em GET /api/servicos, recebe contatos em POST /api/contato, documenta a API em /docs e expõe o health check /health. O código é empacotado em um container Docker e preparado para o Azure App Service. O modelo principal é PaaS: a plataforma opera infraestrutura, host, roteamento, TLS e escala; a equipe mantém imagem, código, configurações e dados.

NuvemLab - arquitetura simplificada

Entrega automatizada e execução de um único container no Azure App Service (PaaS)

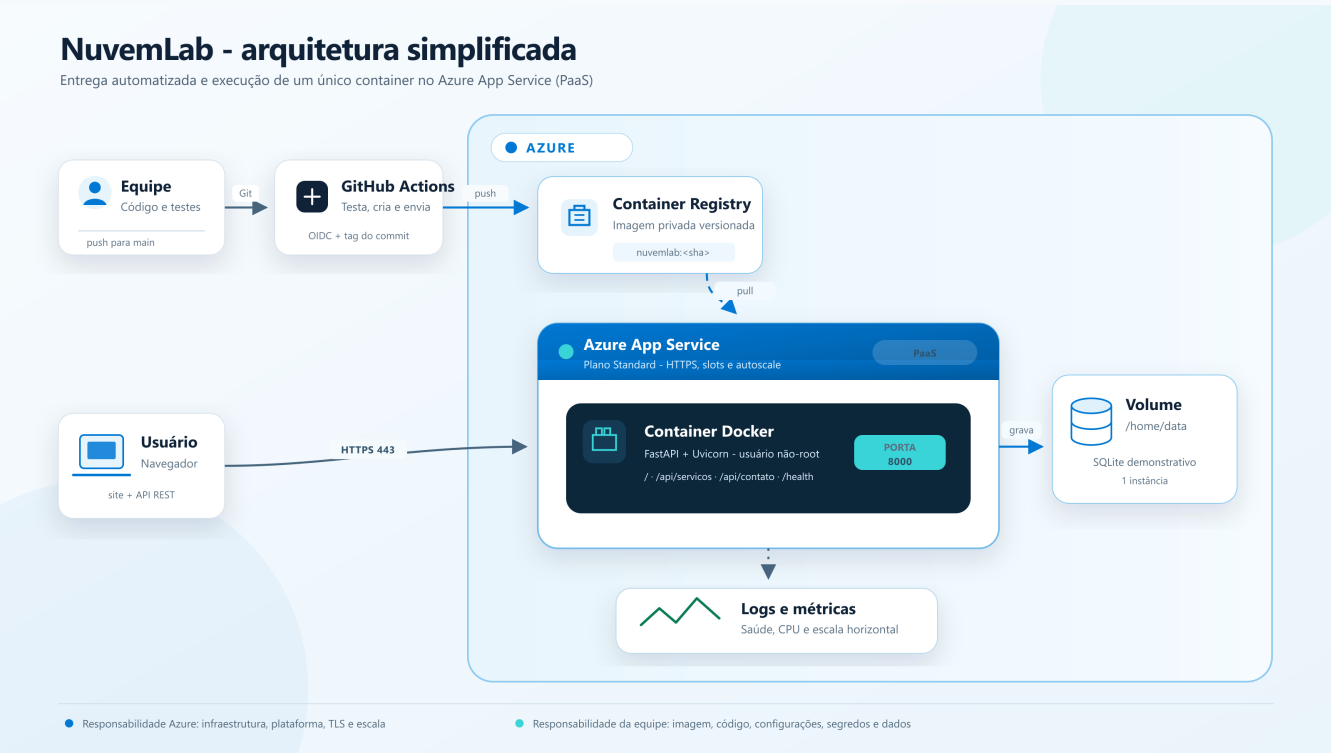


Figura 1 - Fluxos de entrega e execução. O SQLite demonstra o volume persistente com uma instância.

Decisão arquitetural

PaaS + FastAPI + Docker + Azure App Service. A combinação mantém a aplicação portátil e reduz o trabalho operacional sem abrir mão do controle sobre o código e a imagem.

1. Planejamento da arquitetura

Escolha e justificativa do modelo de serviço

Foi escolhido PaaS, representado pelo Azure App Service. Em IaaS, a equipe administraria máquinas virtuais, sistema operacional, patches e parte da rede, trabalho desproporcional a uma aplicação pequena. SaaS não se aplica porque o objetivo é desenvolver software próprio. XaaS é um termo abrangente, porém menos preciso para a divisão de responsabilidades adotada. O container personalizado adiciona portabilidade, mas a plataforma gerenciada continua sendo o serviço principal.

Modelo	Avaliação	Justificativa
IaaS	Não adotado	Maior controle, mas exige gestão de VM, SO e patches.
PaaS	Adotado	Foco no código; runtime, TLS, roteamento e escala gerenciados.
SaaS	Não aplicável	Seria consumir uma solução pronta, não implantar a aplicação.
XaaS	Conceito geral	Útil como categoria ampla, mas não descreve a escolha com precisão.

Componentes usados

Componente	Papel na solução
FastAPI + Uvicorn	Página, API REST, validação, OpenAPI e servidor ASGI.
Jinja2 + assets locais	HTML responsivo, estilos e integração do formulário.
SQLite	Persistência demonstrativa de contatos em uma instância.
Docker	Imagem reproduzível baseada em Python oficial.
Docker Compose	Rede, porta, volume e políticas locais de segurança.
Azure Container Registry	Registro privado com imagens versionadas por commit.
Azure App Service	PaaS Linux para HTTPS, slots, métricas e escala.
GitHub Actions	Testes, build, push, smoke test e promoção.

Modo de hospedagem

O relatório assume um único container Linux no modo clássico do App Service. Na configuração moderna com sidecars, a porta é definida por target-port, e não por WEBSITES_PORT.

Fluxo de execução

O usuário acessa a borda HTTPS do App Service. A plataforma encaminha a requisição à porta 8000 do container. O FastAPI processa a página e os endpoints; contatos válidos são gravados em /home/data/contatos.db. O endpoint /health valida a aplicação e a abertura do banco.

2. Preparação do ambiente com Docker

Dockerfile configurado

A imagem usa python:3.13-slim e dois estágios. O primeiro cria o ambiente virtual e instala dependências; o segundo recebe somente o runtime e o código. O processo executa com UID/GID 10001, sem root. O EXPOSE documenta a porta, enquanto o mapeamento real é feito pelo Compose ou pela plataforma.

Elemento	Implementação	Resultado
Imagem oficial	python:3.13-slim	Base pequena e mantida pelo projeto Python.
Multi-stage	builder + runtime	Dependências isoladas da etapa de construção.
Privilégios	USER app (10001)	Reduz o impacto de uma exploração.
Porta	EXPOSE 8000	Contrato explícito; não publica a porta sozinho.
Inicialização	Uvicorn em 0.0.0.0:8000	Aceita o tráfego encaminhado pela plataforma.
Proxy	--proxy-headers	Interpreta os cabeçalhos da terminação TLS.
Saúde	GET /health	Verifica API e acesso ao SQLite.
Arquivos	.dockerignore	Exclui caches, Git, banco, testes e segredos.

Simulação de rede, porta e volume persistente

Requisito	Configuração no docker-compose.yml	Evidência
Rede	bridge nomeada nuvemlab_net	Isolamento e resolução do serviço web.
Porta	127.0.0.1:8000:8000	Acesso local sem expor em todas as interfaces.
Volume	nuvemlab_contatos_data:/home/data	O SQLite permanece após restart.

Controles de segurança

- Filesystem somente leitura, exceto volume e tmpfs em /tmp.
- cap_drop: ALL e no-new-privileges no Compose.
- Validação Pydantic de formato e tamanho; SQL parametrizado.
- CSP, nosniff, proteção contra frames e política de referenciador.
- Segredos não entram na imagem nem no repositório.

Execução e comprovação

O comando docker compose up --build cria todos os recursos. Depois de enviar um contato, docker compose restart reinicia o serviço sem apagar o volume; a contagem no SQLite permanece. O comando docker compose down encerra o ambiente e preserva o volume por padrão.

```
docker compose up --build
docker compose exec web python -c "import sqlite3;
print(sqlite3.connect('/home/data/contatos.db').execute('select count(*) from
contatos').fetchone()[0])"
```

3. Simulação de deploy no Azure

Estratégia automatizada (CI/CD)

A entrega selecionada combina GitHub Actions, Azure Container Registry e Azure App Service. A autenticação usa OpenID Connect (OIDC), evitando publish profile e senhas permanentes do registro.

Etapa	Ação e critério de aceite
1. Validar	Instala dependências; Ruff e seis testes precisam passar.
2. Autenticar	GitHub obtém token temporário do Azure por OIDC.
3. Construir	Docker cria a imagem e aplica a tag imutável github.sha.
4. Publicar	A imagem é enviada ao ACR privado.
5. Staging	O slot recebe a nova tag e inicia o container.
6. Testar	O pipeline exige resposta 200 de GET /health.
7. Promover	Swap aquece e move staging para production.
8. Reverter	Novo swap restaura a versão anterior se necessário.

Configurações do App Service

App setting	Valor	Finalidade
WEBSITES_PORT	8000	Roteia a única porta HTTP do container clássico.
PORT	8000	Configura a porta de escuta do Uvicorn.
WEBSITES_ENABLE_APP_SERVICE_STORAGE	true	Habilita a persistência compartilhada em /home.
DATA_DIR	/home/data	Mantém o código em /app e somente os dados em /home.

Plataforma, identidade e segurança

- Plano Standard: fornece slots e Azure Monitor Autoscale por métricas.
- ACR privado: App Service usa identidade gerenciada; em RBAC tradicional, recebe AcrPull.
- TLS: termina na borda do App Service; habilitar HTTPS Only, TLS mínimo e logs do container.
- Slots: identidades e rede não são trocadas; staging também precisa de permissão para baixar a imagem.

Limite consciente

SQLite em /home demonstra persistência com uma instância. Antes do scale-out de produção, os contatos migram para Azure SQL ou Azure Database for PostgreSQL, mantendo os containers sem estado.

4. Benefícios, desafios e conceitos de nuvem

Benefícios e desafios

Benefício	Desafio relacionado	Tratamento
Menor carga operacional	Dependência do provedor	Container padrão e configuração documentada.
Deploy repetível	Falha de uma versão	Staging, smoke test, tag imutável e rollback.
TLS e identidade gerenciados	Segredos e permissões	OIDC e privilégio mínimo.
Escala horizontal	Estado compartilhado	Banco gerenciado antes de múltiplas instâncias.
Observabilidade	Custo do plano	Métricas, orçamento, alertas e limites de escala.

Escalabilidade, elasticidade e responsabilidade compartilhada

Escalabilidade é a capacidade de aumentar recursos ou instâncias. Elasticidade é ajustar essa capacidade automaticamente conforme métricas e limites. O scale-out do App Service fornece escalabilidade; as regras do Azure Monitor Autoscale acrescentam elasticidade.

Responsabilidade Azure	Responsabilidade da equipe
Datacenter, hardware, virtualização e host	Código, lógica de negócio e testes
Plataforma, patches gerenciados, roteamento e TLS	Dockerfile, imagem e dependências
Mecanismo de escala, slots e métricas	Dados, retenção, variáveis e segredos
Disponibilidade do serviço contratado	Alertas, incidentes, permissões e custos

Validação e evidências

Verificação	Resultado
Pytest	6 testes: página, docs, catálogo, persistência, validação e health check.
Ruff	Código Python sem erros de lint.
Interface	Sem erros de console; formulário retornou 201 Created.
Diagrama	SVG válido e PNG renderizado sem cortes ou sobreposições.
Relatório	PDF final com exatamente cinco páginas.

Conclusão

A proposta atende ao enunciado com uma solução executável, segura e reproduzível. PaaS reduz o trabalho indiferenciado de infraestrutura; Docker padroniza a execução; CI/CD e slots reduzem o risco de entrega. O SQLite cumpre a demonstração de volume, e a migração prevista para um banco gerenciado completa o caminho para uma produção realmente escalável.

Referências oficiais

1. Microsoft Learn. [Configure a custom container for Azure App Service.](#)
2. Microsoft Learn. [Deploy a container with GitHub Actions.](#)
3. Microsoft Learn. [Set up staging environments.](#)
4. Microsoft Learn. [Scale up and out in App Service.](#)
5. Docker Docs. [Dockerfile reference: EXPOSE.](#)
6. FastAPI. [FastAPI in Containers - Docker.](#)